DBMS | CASE STUDY REPORT

Automating Book Circulation at a University Central Library

A relational database for catalogue, copies, members, loans and fines

STUDENT	Abhinav Yadav
REGISTER NUMBER	24BCE2968
COURSE	Database Management Systems
FACULTY	_____
SLOT	_____
SUBMITTED ON	October 2026

EXECUTIVE SUMMARY

The central library of a mid-sized university lends around 300 books a week to students, faculty and members of the public. The library in this case study is a composite example. Circulation is recorded in a handwritten ledger, so staff cannot tell which copies are on the shelf, which members are over their limit, or who owes fines. Late fines are worked out by hand and are often missed.

This report presents a database that replaces the ledger. Ten relations, designed from an ER model and normalised to BCNF, hold the catalogue, physical copies, members, staff, loans and fines. Five triggers enforce the lending policy: a copy must be available, members must stay within their borrowing limit, members with unpaid fines are blocked, and late fines are created automatically at Rs 10 per day. A console program and a web portal (Athenaeum, a JSON API with a single-page front end) let staff use the system, and 17 SQL queries answer the librarian's questions.

Table 1. Project at a glance

Metric	Value
Relations	11 (incl. materialised view)
Triggers	8
Views	3
Indexes	6 (incl. partial + expression)
Report queries	22
Policy tests passed	11 / 11see Chapter 10

TABLE OF CONTENTS

Chapter 1 Organisation and current process

Chapter 2 Pain points

Chapter 3 Proposed solution and requirements

Chapter 4 Data modelling

Chapter 5 From ER diagram to tables

Chapter 6 Normalisation

Chapter 7 Enforcing the lending policy

Chapter 8 The application

Chapter 9 SQL workload

Chapter 10 Advanced database features

Chapter 11 Validation

Chapter 12 Findings, limitations and next steps

Bibliography

CHAPTER 1 | ORGANISATION AND CURRENT PROCESS

The library holds titles in Computer Science, Databases, Mathematics, History and Fiction, with several physical copies of popular textbooks. Three kinds of member borrow books, each with a different limit:

Table 2. Member categories

Member type	Borrowing limit	Typical use
Student	3 books	Textbooks for the semester
Faculty	8 books	Reference and research
Public	2 books	General reading

1.1 How a loan works today

1. The member brings a book to the counter.
2. A staff member writes the member name, book title and date in the ledger.
3. The due date is written on a slip inside the book (14 days later).
4. On return, staff search the ledger for the entry and strike it out.
5. If the book is late, staff should work out a fine, but this is often forgotten.

CHAPTER 2 | PAIN POINTS

Table 3. Problems with the ledger system

#	Problem observed	Consequence
P1	No live view of which copies are on the shelf	Members queue for books that are already out
P2	Limits are not checked at the counter	Some students hold 5 or 6 textbooks
P3	Fines are calculated by hand	Lost revenue, and disputes with members
P4	A book's title and authors are rewritten on every ledger line	Spelling variations; impossible to count loans per title
P5	No record of which staff member issued a book	No accountability for lost books

CHAPTER 3 | PROPOSED SOLUTION AND REQUIREMENTS

The proposal is a relational database at the centre of circulation, with a simple console program at the counter. The policy rules move from staff memory into the database, so they apply to every transaction.

3.1 What the system must do

Table 4. Requirements traced to pain points

Code	Requirement	Solves
R1	Search the catalogue by title or author and show copies available	P1
R2	Issue a copy to a member, recording the staff member and the due date	P5
R3	Refuse a loan if the copy is out, the member is at their limit, or has unpaid fines	P2, P3
R4	On return, free the copy and add a fine automatically if it is late	P3
R5	Store title, publisher and authors once, and link copies to them	P4
R6	Report overdue loans, popular titles, fines collected and outstanding	P1, P3

3.2 Assumptions

- A book is identified by its 13-digit ISBN; each physical copy has its own copy number.
- The standard loan period is 14 days and the fine is Rs 10 per day late.
- A loan generates at most one fine.

CHAPTER 4 | DATA MODELLING

The ER diagram below uses Chen notation: rectangles are entities, diamonds are relationships, ellipses are attributes (key attributes are underlined), and a double rectangle marks a weak entity.

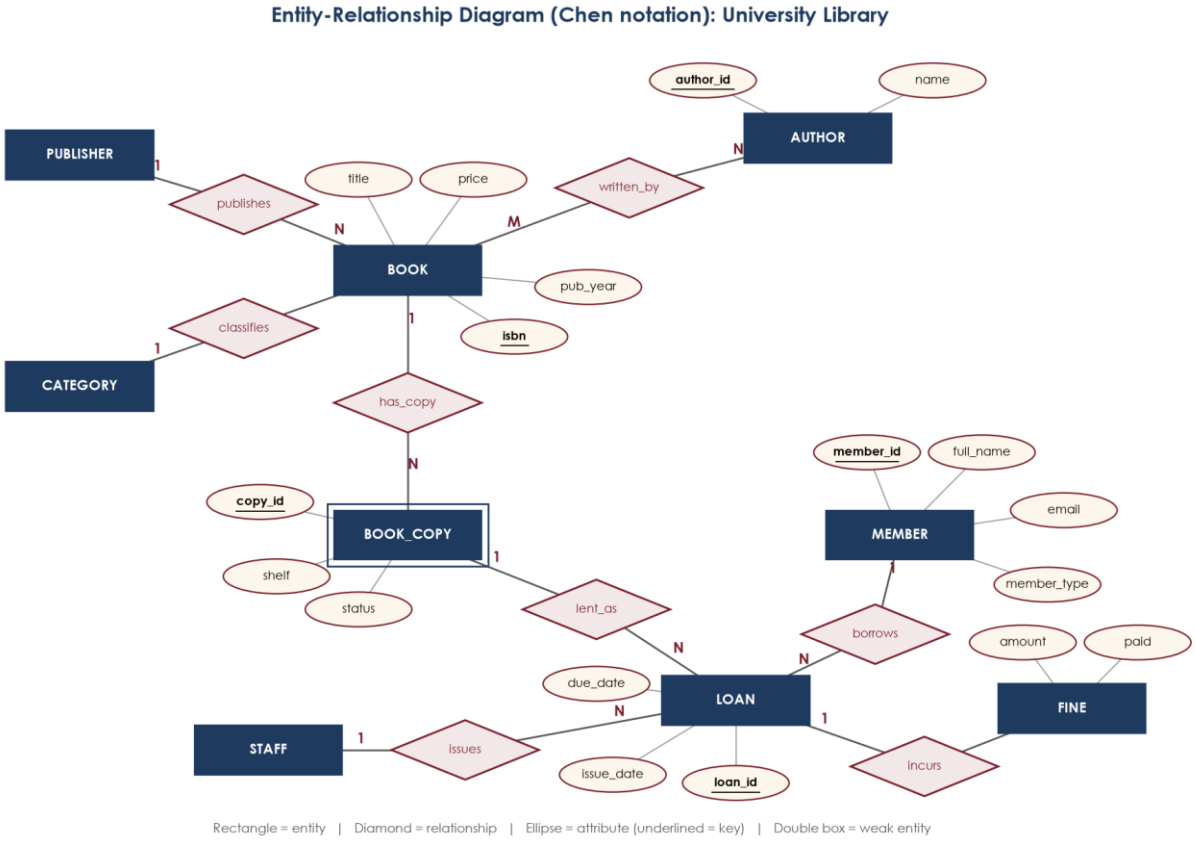


Figure 1. ER diagram of the library (Chen notation)

4.1 Entities

Table 5. Entity catalogue

Entity	Meaning	Key
BOOK	A title in the catalogue	isbn
BOOK_COPY	One physical copy on a shelf (weak, depends on BOOK)	copy_id
AUTHOR	A person who wrote one or more books	author_id
PUBLISHER	Publishing house	publisher_id
CATEGORY	Subject area	category_id
MEMBER	Someone allowed to borrow	member_id
STAFF	Library employee who issues books	staff_id
LOAN	One borrowing of one copy	loan_id
FINE	Penalty for a late loan	fine_id

4.2 Why BOOK and BOOK_COPY are separate

This was the key modelling decision. A title (ISBN 9780078022159, Database System Concepts) is one thing; the three copies on shelves DB-01 and DB-02 are three separate things. Loans are made against copies, not titles. If they were merged, the title and authors would be repeated for each copy, and it would be impossible to know which copy a member has.

CHAPTER 5 | FROM ER DIAGRAM TO TABLES

Mapping steps:

- **Strong entities** (BOOK, AUTHOR, PUBLISHER, CATEGORY, MEMBER, STAFF) → one table each.
- **Weak entity** BOOK_COPY → its own table with isbn as a foreign key to its owner.
- **1:N relationships** (publishes, classifies, has_copy, borrows, issues, lent_as) → a foreign key on the N side.
- **M:N relationship** written_by → new table BOOK_AUTHOR(isbn, author_id) with a composite key.
- **1:0..1 relationship** incurs → FINE.loan_id with a UNIQUE constraint.

The resulting tables, read from the implemented database:

Table 6. BOOK

Attribute	Type	Constraint
isbn	CHAR(13)	PK
title	VARCHAR(200)	NOT NULL
publisher_id	INTEGER	FK -> publisher(publisher_id), NOT NULL
category_id	INTEGER	FK -> category(category_id), NOT NULL
pub_year	INTEGER	
price	DECIMAL(8,2)	NOT NULL

Table 7. BOOK_COPY

Attribute	Type	Constraint
copy_id	INTEGER	PK
isbn	CHAR(13)	FK -> book(isbn), NOT NULL
shelf	VARCHAR(10)	NOT NULL
status	VARCHAR(12)	NOT NULL, DEFAULT 'AVAILABLE'

Table 8. BOOK_AUTHOR

Attribute	Type	Constraint
isbn	CHAR(13)	PK, FK -> book(isbn)
author_id	INTEGER	PK, FK -> author(author_id)
author_order	INTEGER	NOT NULL, DEFAULT 1

Table 9. AUTHOR

Attribute	Type	Constraint
author_id	INTEGER	PK
first_name	VARCHAR(50)	NOT NULL
last_name	VARCHAR(50)	NOT NULL
nationality	VARCHAR(50)	

Table 10. PUBLISHER

Attribute	Type	Constraint
publisher_id	INTEGER	PK
name	VARCHAR(100)	NOT NULL
city	VARCHAR(60)	
phone	VARCHAR(20)	

Table 11. CATEGORY

Attribute	Type	Constraint
category_id	INTEGER	PK
name	VARCHAR(60)	NOT NULL

Table 12. MEMBER

Attribute	Type	Constraint
member_id	INTEGER	PK
full_name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	NOT NULL
phone	VARCHAR(20)	
member_type	VARCHAR(10)	NOT NULL
join_date	DATE	NOT NULL, DEFAULT DATE('now')
max_books	INTEGER	NOT NULL, DEFAULT 3

Table 13. STAFF

Attribute	Type	Constraint
staff_id	INTEGER	PK
full_name	VARCHAR(100)	NOT NULL
role	VARCHAR(30)	NOT NULL

Table 14. LOAN

Attribute	Type	Constraint
loan_id	INTEGER	PK
copy_id	INTEGER	FK -> book_copy(copy_id), NOT NULL
member_id	INTEGER	FK -> member(member_id), NOT NULL
staff_id	INTEGER	FK -> staff(staff_id), NOT NULL
issue_date	DATE	NOT NULL, DEFAULT DATE('now')
due_date	DATE	NOT NULL
return_date	DATE	

Table 15. FINE

Attribute	Type	Constraint
fine_id	INTEGER	PK
loan_id	INTEGER	FK -> loan(loan_id), NOT NULL
amount	DECIMAL(8,2)	NOT NULL
paid	INTEGER	NOT NULL, DEFAULT 0

CHAPTER 6 | NORMALISATION

A single ledger line is effectively this relation:

```
LEDGER(member_id, member_name, email, isbn, title, authors, publisher, publisher_city,
       category, copy_id, shelf, issue_date, due_date, return_date, fine)
```

Table 16. Normalisation walk-through

Step	Problem found	Action taken
1NF	'authors' holds a list (e.g. Silberschatz, Korth, Sudarshan)	One row per author, giving BOOK_AUTHOR
2NF	In BOOK_AUTHOR(isbn, author_id, title...), title depends on isbn only (partial dependency)	Move title, price and year to BOOK
3NF	isbn → publisher → publisher_city (transitive dependency)	PUBLISHER table; same for CATEGORY
3NF	copy_id → isbn → title (transitive)	BOOK_COPY keeps only isbn, shelf, status
3NF	loan → member_id → member_name, email	MEMBER table; LOAN keeps only member_id
BCNF	member: email → all attributes	email is a candidate key (UNIQUE), so BCNF still holds

Final functional dependencies, each with a key on the left:

```
isbn      -> title, publisher_id, category_id, pub_year, price
copy_id   -> isbn, shelf, status
member_id -> full_name, email, phone, member_type, join_date, max_books
email     -> member_id    (candidate key)
loan_id   -> copy_id, member_id, staff_id, issue_date, due_date, return_date
loan_id   -> fine_id, amount, paid    (via UNIQUE fine.loan_id)
```

CHAPTER 7 | ENFORCING THE LENDING POLICY

CHECK constraints cover rules that involve a single row: price ≥ 0, due_date > issue_date, member_type IN ('STUDENT','FACULTY','PUBLIC'), and the four allowed copy statuses. Rules that need other tables are written as triggers:

Table 17. Lending-policy triggers

Trigger	When	Rule
trg_loan_check_copy	BEFORE INSERT on loan	Copy status must be AVAILABLE
trg_loan_check_limit	BEFORE INSERT on loan	Open loans must be below member.max_books
trg_loan_check_fines	BEFORE INSERT on loan	Member must have no unpaid fine
trg_loan_issue	AFTER INSERT on loan	Set copy status to ON_LOAN
trg_loan_return	AFTER UPDATE of return_date	Set copy AVAILABLE; insert fine if late

The return trigger is the most interesting one. It works out the fine with date arithmetic and inserts it only when the book is late:

```
CREATE TRIGGER trg_loan_return
AFTER UPDATE OF return_date ON loan
WHEN OLD.return_date IS NULL AND NEW.return_date IS NOT NULL
BEGIN
    UPDATE book_copy SET status = 'AVAILABLE' WHERE copy_id = NEW.copy_id;
    INSERT INTO fine (loan_id, amount)
    SELECT NEW.loan_id, (JULIANDAY(NEW.return_date) - JULIANDAY(NEW.due_date)) * 10
    WHERE JULIANDAY(NEW.return_date) > JULIANDAY(NEW.due_date);
END;
```

7.1 Views

- **v_book_availability:** total and available copies per title.
- **v_overdue_loans:** open loans past their due date, with days overdue.
- **v_member_summary:** loans, active loans and unpaid fines per member.

CHAPTER 8 | THE APPLICATION

Staff use a menu-driven console program (library.py) written in Python with the built-in sqlite3 module. Each menu option maps to one or more SQL statements. Issuing a book is wrapped in a transaction: if any trigger raises an error, nothing is saved, and the message is shown to the operator.

Table 18. Menu options

Option	What it does	SQL feature used
1 Search books	Keyword search on title/author	LEFT JOIN, LIKE, GROUP_CONCAT
2 Book availability	Copies on shelf per title	View v_book_availability
3 Add member	Register a member	INSERT, UNIQUE email
4 Issue a book	Lend first free copy	INSERT + 3 BEFORE triggers
5 Return a book	Close a loan	UPDATE + AFTER trigger
6 Show open loans	All unreturned loans	CASE for OVERDUE/OK
7 Pay fines	Mark fines paid	UPDATE in a transaction
8 Member summary	Per-member statistics	View v_member_summary
9 Run all report queries	Prints the 17 queries	Runs sql/03_queries.sql

8.1 Web portal: Athenaeum

For the front office and for members, a second interface was built as a web portal (portal.py + portal.html). Unlike the console program, it is split into two tiers: the Python server exposes a small JSON API (/api/books, /api/loans, /api/members, /api/issue, /api/return, /api/pay), and the page itself is a single-page application that calls the API with fetch(). Both interfaces share the same library.db, so the triggers in Chapter 7 protect the portal exactly as they protect the console.

Table 19. Portal screens

Screen	What it shows	Endpoint / SQL
Overview	Counts, fines due, most-borrowed titles, books out	/api/stats: aggregates + view v_overdue_loans
Catalogue	Book cards coloured by subject, live search, copies on shelf	/api/books: 5-table join + HAVING ... LIKE
Circulation Desk	Issue a title, return open loans	POST /api/issue, /api/return (one transaction each)
Members	Register; click a member for full loan and fine history	/api/member: loan LEFT JOIN fine

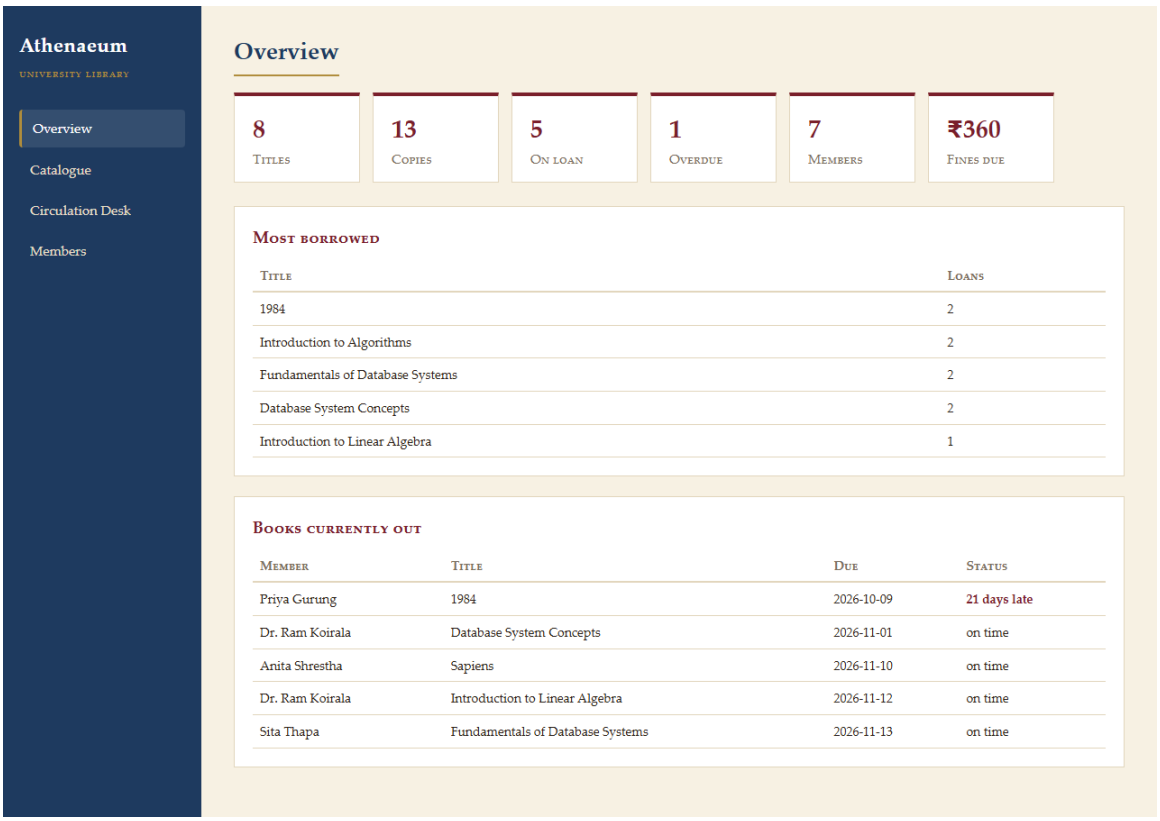


Figure 2. Portal overview

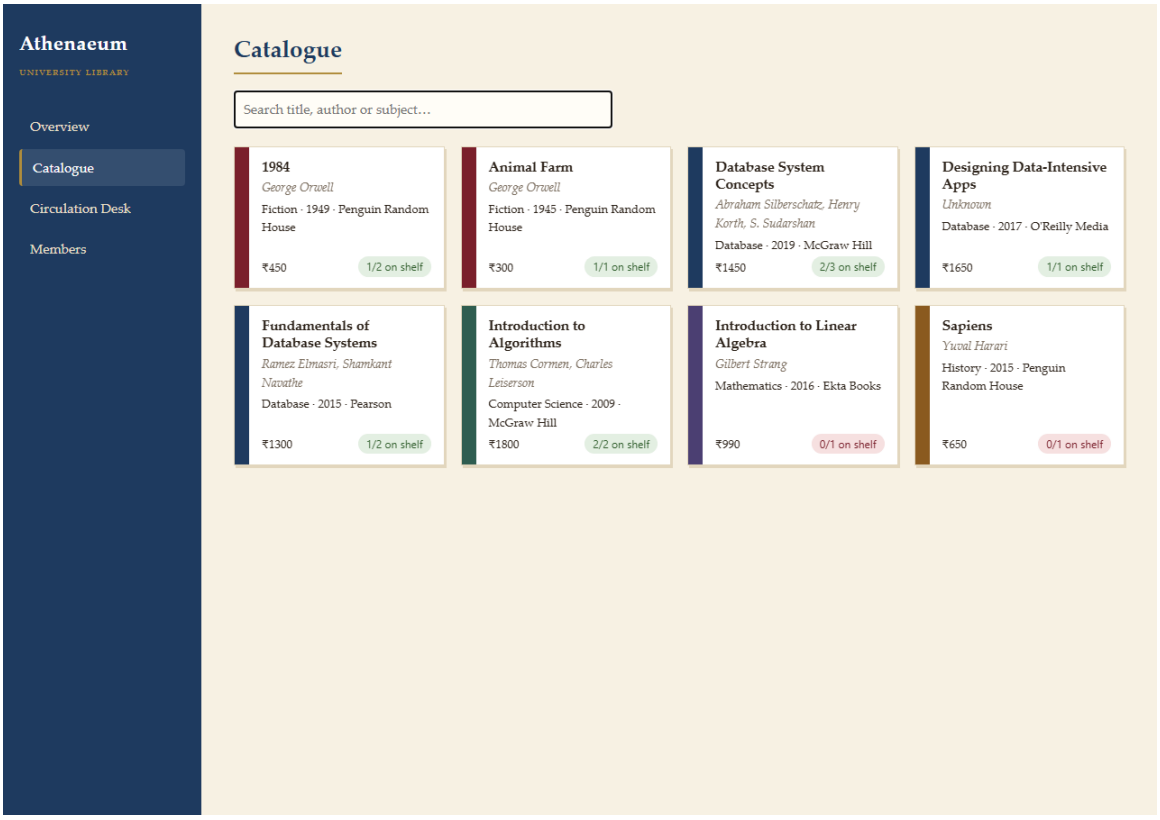


Figure 3. Catalogue: each card's spine colour is the book's category

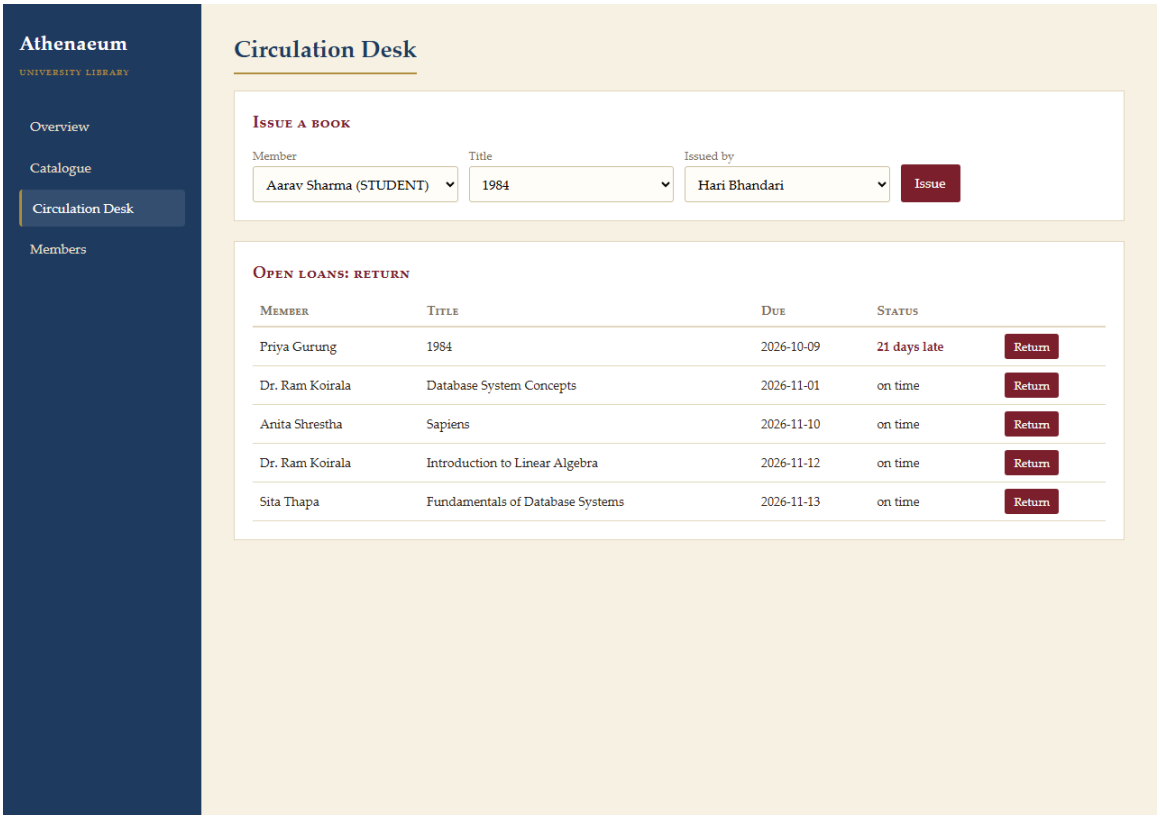


Figure 4. Circulation desk: refusals from triggers appear as messages

8.2 Console program screens

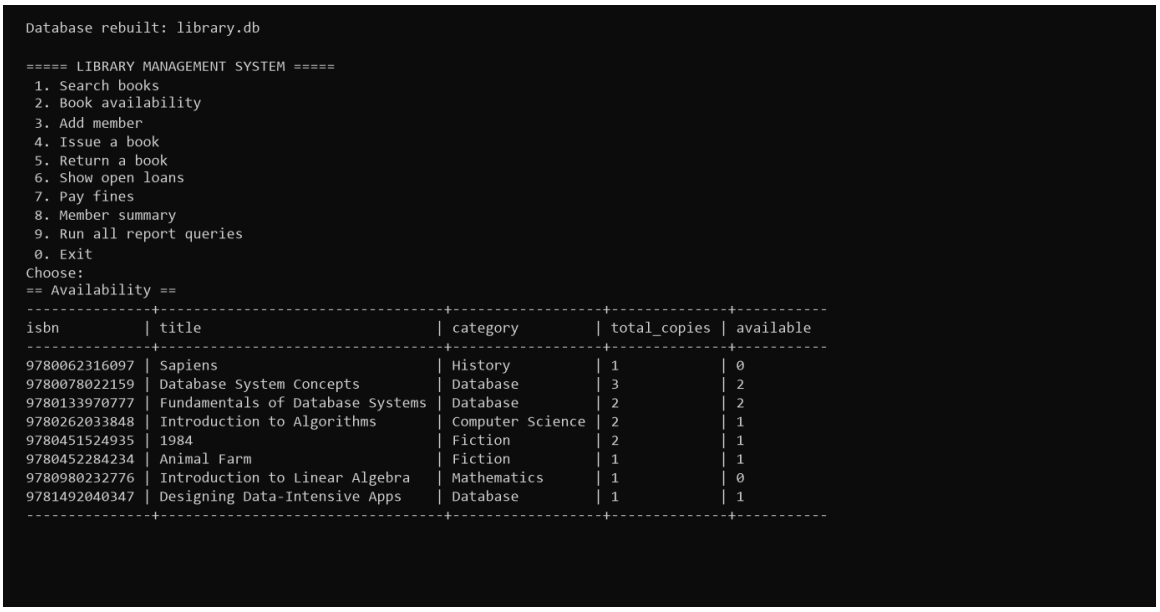
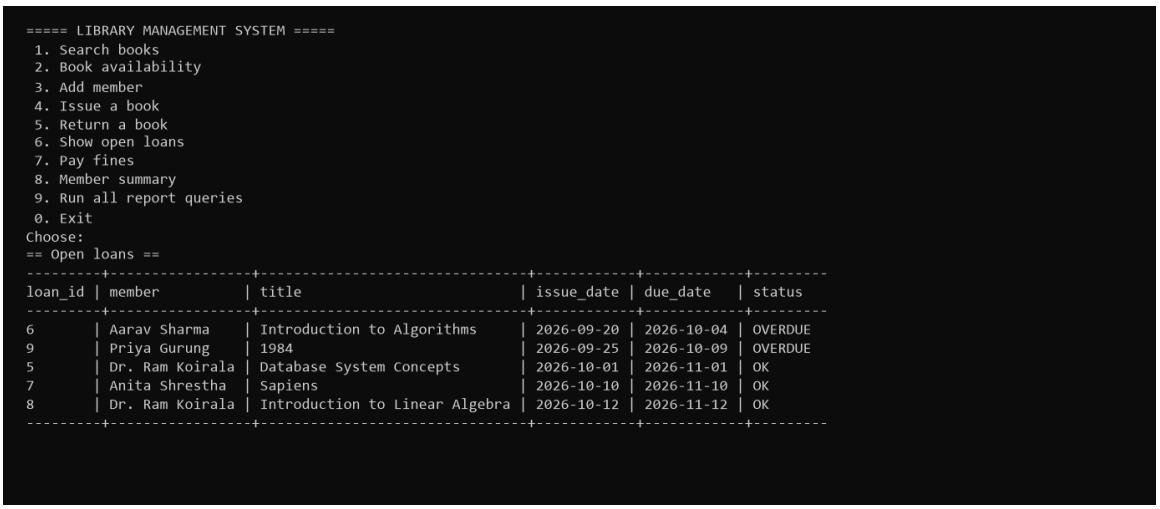


Figure 5. Main menu and the book availability report



```
===== LIBRARY MANAGEMENT SYSTEM =====
 1. Search books
 2. Book availability
 3. Add member
 4. Issue a book
 5. Return a book
 6. Show open loans
 7. Pay fines
 8. Member summary
 9. Run all report queries
 0. Exit
Choose:
== Open loans ==
---------+----------------+-------------------------------+------------+------------+---------
loan_id | member          | title                         | issue_date | due_date   | status
---------+----------------+-------------------------------+------------+------------+---------
6       | Aarav Sharma    | Introduction to Algorithms    | 2026-09-20 | 2026-10-04 | OVERDUE
9       | Priya Gurung    | 1984                          | 2026-09-25 | 2026-10-09 | OVERDUE
5       | Dr. Ram Koirala | Database System Concepts      | 2026-10-01 | 2026-11-01 | OK
7       | Anita Shrestha  | Sapiens                       | 2026-10-10 | 2026-11-10 | OK
8       | Dr. Ram Koirala | Introduction to Linear Algebra | 2026-10-12 | 2026-11-12 | OK
---------+----------------+-------------------------------+------------+------------+---------
```

Figure 6. Open loans with overdue status

CHAPTER 9 | SQL WORKLOAD

The librarian's questions were translated into queries. Each is shown with its result on the sample data (8 titles, 13 copies, 7 members, 9 loans).

9.1 Which books, with all their authors?

Inner join across four tables, with GROUP_CONCAT to list co-authors.

```
SELECT b.title, GROUP_CONCAT(a.first_name || ' ' || a.last_name, ', ') AS authors,
       p.name AS publisher
FROM book b
JOIN book_author ba ON ba.isbn = b.isbn
JOIN author a       ON a.author_id = ba.author_id
JOIN publisher p    ON p.publisher_id = b.publisher_id
GROUP BY b.isbn ORDER BY b.title;
```

Table 20. Result 1

title	authors	publisher
1984	George Orwell	Penguin Random House
Animal Farm	George Orwell	Penguin Random House
Database System Concepts	Abraham Silberschatz, Henry Korth, S. Sudarshan	McGraw Hill
Fundamentals of Database Systems	Ramez Elmasri, Shamkant Navathe	Pearson
Introduction to Algorithms	Thomas Cormen, Charles Leiserson	McGraw Hill
Introduction to Linear Algebra	Gilbert Strang	Ekta Books
Sapiens	Yuval Harari	Penguin Random House

9.2 Which titles have no author on record?

LEFT OUTER JOIN keeps unmatched books; IS NULL picks them out.

```
SELECT b.isbn, b.title FROM book b
LEFT JOIN book_author ba ON ba.isbn = b.isbn
WHERE ba.author_id IS NULL;
```

Table 21. Result 2

isbn	title
9781492040347	Designing Data-Intensive Apps

9.3 Which categories own more than two copies?

GROUP BY with a HAVING filter on the aggregate.

```
SELECT c.name AS category, COUNT(bc.copy_id) AS copies
FROM category c JOIN book b ON b.category_id = c.category_id
JOIN book_copy bc ON bc.isbn = b.isbn
GROUP BY c.name HAVING COUNT(bc.copy_id) > 2;
```

Table 22. Result 3

category	copies
Database	6
Fiction	3

9.4 Most expensive book in each category

Correlated sub-query: the inner query is re-evaluated for each outer row.

```
SELECT c.name AS category, b.title, b.price
FROM book b JOIN category c ON c.category_id = b.category_id
WHERE b.price = (SELECT MAX(b2.price) FROM book b2 WHERE b2.category_id = b.category_id);
```

Table 23. Result 4

category	title	price
Computer Science	Introduction to Algorithms	1800
Fiction	1984	450
History	Sapiens	650
Mathematics	Introduction to Linear Algebra	990
Database	Designing Data-Intensive Apps	1650

9.5 Members who have never borrowed

NOT EXISTS anti-join.

```
SELECT m.member_id, m.full_name FROM member m
WHERE NOT EXISTS (SELECT 1 FROM loan l WHERE l.member_id = m.member_id);
```

Table 24. Result 5

member_id	full_name
7	Kiran Magar

9.6 Titles that have never been borrowed

Set difference with EXCEPT.

```
SELECT b.isbn, b.title FROM book b
EXCEPT
SELECT b.isbn, b.title FROM book b
JOIN book_copy bc ON bc.isbn = b.isbn JOIN loan l ON l.copy_id = bc.copy_id;
```

Table 25. Result 6

isbn	title
9780452284234	Animal Farm
9781492040347	Designing Data-Intensive Apps

9.7 Which loans are overdue right now?

Query on a view.

```
SELECT member, title, due_date, days_overdue FROM v_overdue_loans ORDER BY days_overdue
DESC;
```

Table 26. Result 7

member	title	due_date	days_overdue
Aarav Sharma	Introduction to Algorithms	2026-10-04	26
Priya Gurung	1984	2026-10-09	21

9.8 Member status

View plus CASE to label each member.

```
SELECT full_name, active_loans, unpaid_fines,
       CASE WHEN unpaid_fines > 0 THEN 'BLOCKED'
            WHEN active_loans = 0 THEN 'IDLE' ELSE 'ACTIVE' END AS status
FROM v_member_summary;
```

Table 27. Result 8

full_name	active_loans	unpaid_fines	status
Aarav Sharma	1	0	ACTIVE
Sita Thapa	0	0	IDLE
Dr. Ram Koirala	2	0	ACTIVE
Priya Gurung	1	0	ACTIVE
Bikash Rai	0	100	BLOCKED
Anita Shrestha	1	0	ACTIVE
Kiran Magar	1	0	ACTIVE

9.9 How much has been collected in fines?

Conditional aggregation.

```
SELECT SUM(CASE WHEN paid = 1 THEN amount ELSE 0 END) AS collected,
       SUM(CASE WHEN paid = 0 THEN amount ELSE 0 END) AS outstanding
FROM fine;
```

Table 28. Result 9

collected	outstanding
60	100

CHAPTER 10 | ADVANCED DATABASE FEATURES

Four more advanced techniques were added. Each one solves a specific library need rather than being there just for show.

10.1 Incrementally maintained materialised view

The librarian checks 'most borrowed' many times a day. A normal view re-runs COUNT(*) over every loan each time it is read. BOOK_STATS stores the result instead and is kept current by three triggers: one creates a stats row when a book is catalogued, one adds 1 to times_borrowed on every loan, and one adds late days on a late return. Reads become a primary-key lookup, at the cost of a little extra work on each write. This is the classic read/write trade-off behind materialised views in Oracle and PostgreSQL.

```
CREATE TRIGGER trg_stats_loan AFTER INSERT ON loan
BEGIN
    UPDATE book_stats
       SET times_borrowed = times_borrowed + 1,
           last_borrowed  = MAX(COALESCE(last_borrowed, ''), NEW.issue_date)
     WHERE isbn = (SELECT isbn FROM book_copy WHERE copy_id = NEW.copy_id);
END;
```

Table 29. BOOK_STATS contents

title	times_borrowed	last_borrowed	total_late_days
1984	2	2026-09-25	6
Database System Concepts	2	2026-10-01	0
Introduction to Algorithms	2	2026-09-20	0
Fundamentals of Database Systems	1	2026-09-01	10
Introduction to Linear Algebra	1	2026-10-12	0
Sapiens	1	2026-10-10	0
Animal Farm	0		0
Designing Data-Intensive Apps	0		0

To prove the stored figures are correct, the view is compared with a live aggregate using set difference. An empty result means they match exactly:

```
SELECT isbn, times_borrowed FROM book_stats
EXCEPT
SELECT b.isbn, COUNT(l.loan_id) FROM book b
LEFT JOIN book_copy bc ON bc.isbn = b.isbn LEFT JOIN loan l ON l.copy_id = bc.copy_id
GROUP BY b.isbn;
```

Result: Rows returned: 0 (consistent).

10.2 Recommendations by self-join ('readers also borrowed')

LOAN is joined to itself on member_id to find pairs of loans by the same reader. Grouping by the second book and counting distinct readers gives a simple co-occurrence recommender, the idea behind 'customers also bought', written in plain SQL.

```
SELECT b2.title AS also_borrowed, COUNT(DISTINCT l1.member_id) AS shared_readers
FROM loan l1
JOIN book_copy c1 ON c1.copy_id = l1.copy_id
```

```
JOIN loan l2      ON l2.member_id = l1.member_id AND l2.loan_id <> l1.loan_id
JOIN book_copy c2 ON c2.copy_id = l2.copy_id
JOIN book b2      ON b2.isbn = c2.isbn
WHERE c1.isbn = '9780078022159' AND c2.isbn <> c1.isbn
GROUP BY b2.isbn ORDER BY shared_readers DESC;
```

Table 30. Readers of 'Database System Concepts' also borrowed

also_borrowed	shared_readers
Introduction to Algorithms	2
Introduction to Linear Algebra	1

10.3 Temporal (point-in-time) query

Because each loan stores both issue_date and return_date, the LOAN table is a valid-time table: each row records the period during which a copy was out. That means the database can answer 'what did the shelf look like on any past day' with one predicate, a period-containment test:

```
SELECT b.title, m.full_name AS borrower, l.issue_date, COALESCE(l.return_date, 'not yet') AS
returned
FROM loan l JOIN book_copy bc ON bc.copy_id = l.copy_id JOIN book b ON b.isbn = bc.isbn
JOIN member m ON m.member_id = l.member_id
WHERE l.issue_date <= '2026-08-20' AND (l.return_date IS NULL OR l.return_date > '2026-08-
20');
```

Table 31. Books on loan on 20 Aug 2026

title	borrower	issue_date	returned
1984	Sita Thapa	2026-08-10	2026-08-30
Introduction to Algorithms	Dr. Ram Koirala	2026-08-05	2026-09-01

10.4 Partial and expression indexes

Most loans are closed, but the borrowing-limit trigger only ever looks at open ones. A partial index stores only the rows WHERE return_date IS NULL, so it stays small however many years of history build up. An expression index on LOWER(title) lets case-insensitive title lookups seek instead of scanning. EXPLAIN QUERY PLAN confirms that both are used:

```
CREATE INDEX idx_loan_open_by_member ON loan(member_id) WHERE return_date IS NULL;
CREATE INDEX idx_book_title_lower    ON book(LOWER(title));
```

Table 32. Execution plans

Query	Optimiser plan
SELECT COUNT(*) FROM loan WHERE member_id = 3 AND return_date IS NULL	SEARCH loan USING INDEX idx_loan_open_by_member (member_id=?)
SELECT isbn FROM book WHERE LOWER(title) = 'sapiens'	SEARCH book USING INDEX idx_book_title_lower (<expr>=?)

10.5 In the portal

The Athenaeum portal's Insights screen shows all three read-side features: the popularity bars come from BOOK_STATS, the recommender updates when a title is chosen, and the time machine re-runs the temporal query for any chosen date.

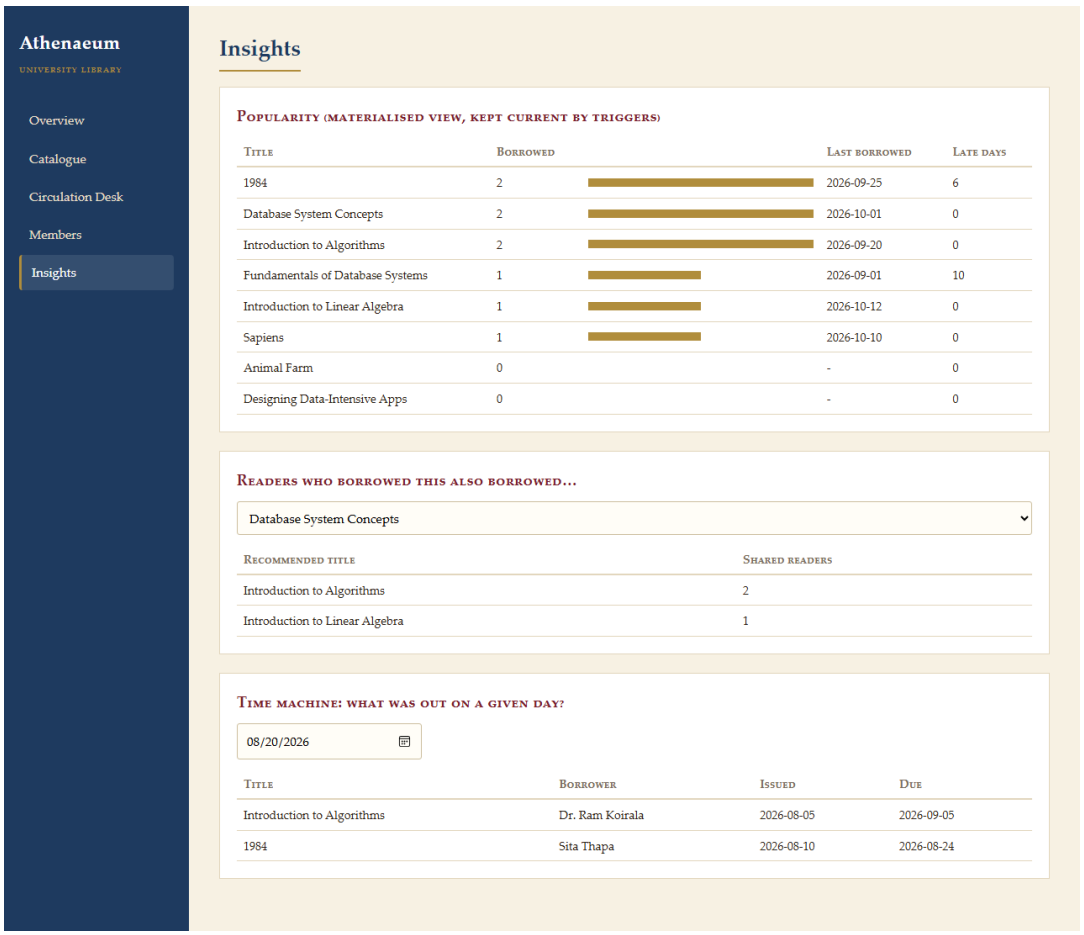


Figure 7. Insights: materialised view, recommender and time machine

CHAPTER 11 | VALIDATION

Each policy rule was tested by attempting a transaction that should be refused, or by checking the side effect of one that should succeed. Tests run on a fresh database and are rolled back afterwards. The 'Observed' column is the database's actual response.

Table 33. Policy validation

#	Scenario	Expected	Observed	OK
V1	Issue copy 7 (already on loan)	Refused	Rejected: Copy is not available for loan	✓
V2	Public member Priya (limit 2) borrows 2 more	Refused on 2nd	Rejected: Member has reached the borrowing limit	✓
V3	Bikash (unpaid Rs 100 fine) borrows	Refused	Rejected: Member has unpaid fines	✓
V4	Issue copy 3 to Sita; check copy status	ON_LOAN	ON_LOAN	✓
V5	Return loan 6 on 2026-10-25 (due 10-04)	Fine Rs 210	210	✓
V6	Return loan 5 on time; count its fines	0	0	✓
V7	Due date before issue date	Refused (CHECK)	Rejected: CHECK constraint failed: due_date > issue_date	✓
V8	Register a member with a duplicate email	Refused (UNIQUE)	Rejected: UNIQUE constraint failed: member.email	✓
V10	Issue copy 3 to Sita; BOOK_STATS count for its title	2 -> 3	3	✓
V11	Catalogue a new book; stats row auto-created	1	1	✓
V9	Delete a publisher that still has books	Refused (FK)	Rejected: FOREIGN KEY constraint failed	✓

Result: 11/11 checks behaved as expected.

CHAPTER 12 | FINDINGS, LIMITATIONS AND NEXT STEPS

12.1 Findings

- Separating titles from physical copies (BOOK vs BOOK_COPY) gives accurate availability. This was the root cause of pain point P1.
- Putting the lending policy in triggers means it is applied the same way to every loan, whether made through the app or directly in SQL.
- Fines are now calculated automatically when a book is returned, which removes pain point P3.
- Normalisation removed the duplicated title and author text that made the ledger hard to count (P4).

12.2 Limitations

- Single-file SQLite suits one counter. Several counters would need a client-server DBMS such as MySQL.
- There are no reservations or holds when every copy is out.
- Staff log-in is not implemented; the staff ID is typed in by hand.

12.3 Next steps

- Add a RESERVATION table with a queue position, and notify the next member when a copy is returned.
- Add a web or mobile front end so members can search the catalogue themselves.
- Let fine rates vary by member type by moving the rate into a MEMBER_TYPE lookup table.

12.4 Closing remarks

Designed from the library's real pain points, a ten-table normalised schema with a small set of triggers replaces the ledger and enforces the lending policy without depending on staff memory. The design, implementation and validation in this report show that the requirements in Chapter 3 are met.

BIBLIOGRAPHY

[1] Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems, 7th edition, Pearson, 2015.

[2] Silberschatz, A., Korth, H. F. and Sudarshan, S., Database System Concepts, 7th edition, McGraw-Hill, 2019.

[3] Date, C. J., An Introduction to Database Systems, 8th edition, Addison-Wesley, 2003.

[4] SQLite Consortium, "SQL As Understood By SQLite", sqlite.org/lang.html.